SENTIMENT ANALYSIS USING LOGISTIC REGRESSION AND SVM

Objective: Develop two machine-learning models to analyse text reviews and classify them as positive or negative. Train Logistic Regression and Support Vector Machine (SVM), then compare their performance.

1. Understanding the Execution Workflow

Step	What happens
Input dataset	Load customer, movie or product reviews with sentiment labels.
Text preprocessing	Clean the text, convert it to lowercase and remove unwanted punctuation.
TF-IDF vectorization	Convert words and documents into numerical feature vectors.
Model training	Train Logistic Regression and SVM using the training data.
Prediction	Predict whether each review expresses positive or negative sentiment.
Evaluation	Compare accuracy, precision, recall, F1-score and confusion matrices.

2. Software Requirements

Use Python with Jupyter Notebook, Anaconda, or Google Colab. Install the required libraries:

```
pip install pandas numpy scikit-learn matplotlib seaborn
```

3. Dataset Preparation

Create a CSV file named **sentiment_dataset.csv** with two columns: **Review** and **Sentiment**. Use Positive and Negative labels consistently.

Review	Sentiment
The product is excellent	Positive
I really enjoyed this movie	Positive
Very poor quality	Negative
The service was terrible	Negative
Amazing experience	Positive
I did not like this product	Negative

Important: These six rows are only an illustrative sample. Use a larger, balanced dataset for meaningful training and evaluation.

4. Python Implementation

Save the following code as **sentiment_analysis.py** in the same folder as **sentiment_dataset.csv**.

```
# SENTIMENT ANALYSIS USING LOGISTIC REGRESSION AND SVM

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix
)

# STEP 1: Load dataset
data = pd.read_csv("sentiment_dataset.csv")
X = data["Review"]
y = data["Sentiment"]

# STEP 2: Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# STEP 3: Convert text into numerical TF-IDF features
tfidf = TfidfVectorizer(lowercase=True, stop_words="english")
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

# STEP 4: Train Logistic Regression
lr_model = LogisticRegression(max_iter=1000)
lr_model.fit(X_train_tfidf, y_train)

# STEP 5: Train SVM
svm_model = LinearSVC()
svm_model.fit(X_train_tfidf, y_train)

# STEP 6: Predict test-set sentiment
lr_pred = lr_model.predict(X_test_tfidf)
svm_pred = svm_model.predict(X_test_tfidf)

# STEP 7: Evaluate both models
print("\\nLOGISTIC REGRESSION RESULTS")
print("Accuracy:", accuracy_score(y_test, lr_pred))
print(classification_report(y_test, lr_pred))

print("\\nSVM RESULTS")
print("Accuracy:", accuracy_score(y_test, svm_pred))
print(classification_report(y_test, svm_pred))

# STEP 8: Display confusion matrices
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
sns.heatmap(confusion_matrix(y_test, lr_pred), annot=True,
            fmt="d", cmap="Blues", ax=axes[0])
axes[0].set_title("Logistic Regression")
axes[0].set_xlabel("Predicted")
axes[0].set_ylabel("Actual")

sns.heatmap(confusion_matrix(y_test, svm_pred), annot=True,
            fmt="d", cmap="Greens", ax=axes[1])
axes[1].set_title("SVM")
axes[1].set_xlabel("Predicted")
axes[1].set_ylabel("Actual")
plt.tight_layout()
plt.show()

# STEP 9: Predict sentiment of new reviews
new_reviews = [
    "This product is excellent and amazing",
    "The product is very poor and disappointing"
]
new_tfidf = tfidf.transform(new_reviews)

print("\\nNEW REVIEW PREDICTIONS")
print("Logistic Regression:", lr_model.predict(new_tfidf))
print("SVM:", svm_model.predict(new_tfidf))
```

5. How to Execute the Program

1. Open Anaconda Navigator and launch Jupyter Notebook, or open Command Prompt.

2. Create a project folder and place sentiment_analysis.py and sentiment_dataset.csv in it.

3. Install the required libraries using the pip command in Section 2.

4. Open the project folder in Jupyter Notebook, or navigate to it in Command Prompt.

5. Run the Python script. In Jupyter, paste the code into cells and execute them in order.

6. Observe the evaluation metrics, confusion matrices and predictions for new reviews.

Command Prompt execution

```
cd path\\to\\your\\project_folder\npython sentiment_analysis.py
```

6. Understanding the Output

Output	Meaning
Accuracy	Proportion of all test reviews classified correctly.
Precision	Of the reviews predicted as a class, the proportion that truly belongs to that class.
Recall	Of the actual reviews in a class, the proportion correctly identified.
F1-score	Harmonic mean of precision and recall.
Confusion matrix	Table showing correct and incorrect predictions for each class.

7. Key Concepts Used

Technique	Role
TF-IDF	Transforms text into numerical features representing the importance of words in documents.
Logistic Regression	Learns a classification boundary and predicts the sentiment class.
SVM (LinearSVC)	Finds a separating hyperplane with a large margin between sentiment classes.
Train-test split	Keeps some data aside to evaluate the models on unseen reviews.

8. Suggested Lab Experiment

Title: Sentiment Analysis Using Logistic Regression and Support Vector Machine (SVM)

Aim: To implement a sentiment classification system using TF-IDF feature extraction and compare Logistic Regression and SVM in classifying textual reviews as positive or negative.

Learning Outcomes

1. Understand text preprocessing and TF-IDF vectorization.
2. Implement Logistic Regression for sentiment classification.
3. Implement SVM for sentiment classification.
4. Evaluate and compare the performance of both models.

Recommendation: Begin with a sufficiently sized, labelled dataset so that both training and test sets contain examples of both sentiment classes. A tiny sample dataset is useful only for demonstrating the file format; it is not suitable for reliable model evaluation.